Big Data - NevWorld

Conocer el pipeline completo

Objetivo del tema

Hasta ahora hemos procesado una partida. En este tema prepararemos el pipeline para recibir varios JSONL, comprobar su integridad, unir sesiones compatibles y registrar cuánto tarda. Después veremos qué cambiaría si fueran miles de archivos.

Procesar **por lotes (batch)** sirve para aplicar el mismo trabajo a un conjunto de datos ya disponibles sin revisar cada archivo manualmente; por ejemplo, reunir las partidas del día en una sola ejecución. El **streaming** procesa los eventos conforme llegan, para actualizar resultados mientras se juega. El procesamiento **distribuido** reparte el trabajo entre varias máquinas cuando hace falta más capacidad. No son tres opciones excluyentes: tanto un lote como un flujo de eventos pueden procesarse de forma distribuida. En este tema trabajaremos por lotes en un solo ordenador.

```text
1 partida     → ejecución manual
varias partidas → procesamiento por lotes
miles de partidas → particiones y procesamiento distribuido
eventos continuos → streaming
```

Procesamiento por lotes

El procesamiento **batch** o por lotes reúne datos disponibles y los procesa en una ejecución. Nuestro lote serán todos los archivos `nevworld_*.jsonl` presentes en `data/raw` cuando se inicia el script.

Copia varias partidas en `data/raw` si están disponibles. Cada una debe conservar su `run_id`; no combines archivos pegando líneas manualmente.

Integridad mediante checksum

Un **checksum** es una huella del contenido de un archivo. Usaremos **SHA-256** para calcularla. Comparar esa huella con una anterior de confianza permite comprobar si el contenido ha cambiado, aunque el archivo conserve su nombre y tamaño.

Ejemplo: recibir archivos de ventas. Una tienda envía su CSV de ventas y la huella calculada antes del envío. La central calcula la huella del archivo recibido y las compara. Si no coinciden, el contenido recibido es distinto y debe revisarse o solicitarse de nuevo. Cambiar una venta de `100` a `900` mantiene el tamaño del archivo, pero cambia su huella. Esto comprueba la integridad de la copia, no que las ventas registradas sean correctas.

Ejemplo: repetir un informe meses después. Una empresa conserva las huellas de los archivos usados para calcular su facturación. Al repetir el análisis, compara las huellas actuales con las guardadas para comprobar si parte de los mismos datos. Si el resultado cambia y las huellas también, sabe que los archivos de entrada han cambiado.

En NevWorld: imagina que hoy el script de análisis calcula un balance de madera de `102`. Más adelante corriges ese script —por ejemplo, su consulta de recursos, no el código del juego— y obtienes `98`. Para investigar la diferencia, necesitas saber si también han cambiado los JSONL. Comparar sus huellas actuales con las guardadas permite comprobarlo: si coinciden, estás usando el mismo contenido y puedes revisar el cambio del análisis; si no coinciden, los datos de entrada también han cambiado. La huella no explica por sí sola la diferencia ni demuestra qué resultado es correcto. Nuestro pipeline guarda esas huellas en `reports/manifest.csv`, pero

todavía no las compara. Conserva una copia del manifiesto antes de repetirlo, porque se sobrescribe.

Crear el pipeline de lotes

En este paso preparamos el trabajo: indicamos dónde buscar los archivos, creamos las carpetas de salida y empezamos a medir el tiempo. Dejamos listas las estructuras que guardarán las tablas aceptadas y el registro de cada archivo.

Crea `scripts/06_pipeline_lotes.py` y añade los cuatro bloques Python en orden. Calcularemos una huella por archivo y comprobaremos `schema_version`: un campo que escribe el juego para identificar la versión del formato de sus eventos. En este ejemplo aceptamos archivos cuyos registros declaran la versión 2.

python

```python
# hashlib viene con Python: sha256 calcula la huella del archivo.
from hashlib import sha256
# Path permite trabajar con rutas de archivos y carpetas.
from pathlib import Path
# Importamos de time un contador para medir duraciones en segundo
s.
from time import perf_counter

import pandas as pd


RAW_DIR = Path('data/raw')
OUTPUT_DIR = Path('data/processed')
REPORTS_DIR = Path('reports')
# Creamos las carpetas de salida si no existen.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
REPORTS_DIR.mkdir(exist_ok=True)
```

```
# Buscamos los JSONL de NevWorld y ordenamos sus nombres; * es la parte variable.
files = sorted(RAW_DIR.glob('nevworld_*.jsonl'))
# Si no hay archivos, detenemos el programa con un mensaje.
if not files:
    raise FileNotFoundError('No hay JSONL en data/raw')

# Lo siguiente prepara el programa para medir cuánto tarda en ejecutarse, almacenar los datos y registrar información sobre los archivos procesados:

start = perf_counter() #Inicia un cronómetro para medir el tiempo de ejecución.

frames = [] # Lista para almacenar los DataFrames.
manifest = [] # Lista para registrar información sobre cada archivo procesado.
```

Leer y registrar cada archivo

Ahora revisamos cada JSONL para decidir si puede formar parte del lote. Calculamos su huella, comprobamos la versión declarada y anotamos el resultado en el manifiesto. Conservamos las tablas aceptadas con su archivo de origen para poder rastrear sus eventos después.

python

```
# Segundo bloque: añádelo debajo del anterior. Revisamos un archivo por vuelta.
for path in files:
    # Leemos los bytes y obtenemos su huella como texto hexadecimal.
    checksum = sha256(path.read_bytes()).hexdigest()
    # lines=True indica que cada línea contiene un evento JSON independiente.
    frame = pd.read_json(path, lines=True)
```

```
    # Aceptamos tablas no vacías donde todas las filas declaran l
a versión 2.
    # eq(2) compara cada valor y all() exige que todos cumplan.
    valid_schema = (
        'schema_version' in frame.columns
        and not frame.empty
        and frame['schema_version'].eq(2).all()
    )

    # append añade una ficha: nombre, tamaño en bytes, filas, hue
lla y estado.
    manifest.append({
        'file': path.name,
        'bytes': path.stat().st_size,
        'rows': len(frame),
        'sha256': checksum,
        # Elegimos la etiqueta según el resultado de la comprobac
ión.
        'status': 'accepted' if valid_schema else 'rejected',
    })

    # Solo conservamos las tablas compatibles para combinarlas.
    if valid_schema:
        # Guardamos el origen de cada fila para poder rastrearla
después.
        frame['source_file'] = path.name
        frames.append(frame)
```

Unir y comprobar las sesiones

En este paso reunimos las tablas aceptadas en una sola tabla de eventos, manteniendo identificada cada partida. Antes de continuar, comprobamos que no haya eventos repetidos: cargar dos veces el mismo evento alteraría los recuentos y las sumas. La pareja `run_id` y `event_index` identifica el evento dentro de su partida; al final ordenamos por ambos campos.

```
python
```

```
# Tercer bloque: fuera del for, sin sangría.
# No podemos unir tablas si no se ha aceptado ninguna.
if not frames:
    raise ValueError('Ningún archivo utiliza el esquema 2')

# Unimos las filas; ignore_index crea un nuevo índice de pandas.
events = pd.concat(frames, ignore_index=True)

# Contamos repeticiones de la pareja que identifica un evento en
una partida.
duplicates = events.duplicated(
    ['run_id', 'event_index']
).sum()

# Nos detenemos para investigar duplicados, en lugar de borrarlo
s.
if duplicates:
    raise ValueError(
        f'Se detectaron {duplicates} eventos duplicados'
    )

# Ordenamos por partida y, dentro de ella, por número de evento.
events = events.sort_values(
    ['run_id', 'event_index']
)
```

Guardar resultados y métricas

Por último, guardamos el resultado para poder reutilizarlo sin repetir la lectura de todos los JSONL. Exportamos los eventos combinados, el inventario de archivos y un resumen con cuántos archivos, partidas y eventos se han procesado y cuánto tiempo ha empleado el programa.

python

```
# Cuarto bloque: guardamos los eventos combinados en Parquet.
```

```
# Esto no regenera los CSV de recursos, necesidades y demás tabla
s.
events.to_parquet(
    OUTPUT_DIR / 'all_events.parquet',
    index=False,
)

# Convertimos la lista de fichas en una tabla y guardamos el inve
ntario.
pd.DataFrame(manifest).to_csv(
    REPORTS_DIR / 'manifest.csv',
    index=False,
)

# Tiempo transcurrido hasta aquí; no incluye escribir el informe
siguiente.
elapsed = perf_counter() - start

# Creamos un resumen de una fila para esta ejecución.
# nunique cuenta partidas distintas; round redondea los segundos.
run_report = pd.DataFrame([{
    'files_found': len(files),
    'files_accepted': len(frames),
    'sessions': events['run_id'].nunique(),
    'events': len(events),
    'seconds': round(elapsed, 3),
}])

# Guardamos el resumen sin el índice interno de pandas.
run_report.to_csv(
    REPORTS_DIR / 'pipeline_run.csv',
    index=False,
)

# Mostramos el mismo resumen como texto en la consola.
print(run_report.to_string(index=False))
```

Se generan tres archivos: `all_events.parquet` con los eventos aceptados, `manifest.csv` con una ficha por archivo y `pipeline_run.csv` con el

resumen de la ejecución. Los informes se sobrescriben al repetir el script; si falla, no confundas salidas antiguas con resultados nuevos.

Ejecutar

Ejecuta el comando siguiente en la terminal desde la raíz de `bigdata-game`, con el entorno virtual activo. Usa JSONL de partidas finalizadas.
El script automatiza el procesamiento del lote, pero todavía lo lanzamos manualmente.

```powershell
python scripts/06_pipeline_lotes.py
```

Qué debes ver al terminar

En la consola verás una cabecera y una sola fila: es el resumen del lote completo. Indica archivos encontrados y aceptados, partidas distintas (`sessions`), eventos combinados y segundos empleados. No se imprime una fila por partida.

Comprueba los tres archivos generados: `reports/manifest.csv` tiene una ficha por JSONL y su estado; `reports/pipeline_run.csv` guarda el resumen de consola; `data/processed/all_events.parquet` contiene los eventos aceptados, identificados por partida (`run_id`) y archivo de origen (`source_file`). Parquet se consulta con pandas o DuckDB, no como texto en el editor.

En el manifiesto, cuenta los archivos `accepted` y suma sus `rows`: deben coincidir con `files_accepted` y `events` del resumen. Comprueba que las

salidas se hayan actualizado; si el script falla, podrían quedar archivos de una ejecución anterior.

Escalabilidad

Mejorar un único ordenador se denomina **escalado vertical**. Repartir almacenamiento y cálculo entre varias máquinas se denomina **escalado horizontal**. Los sistemas Big Data suelen utilizar la segunda estrategia cuando un equipo deja de ser suficiente.

Particiones

Particionar consiste en dividir un conjunto grande según una clave, por ejemplo fecha, `run_id` o `type`. Así cada proceso puede leer solamente la parte necesaria y distintas máquinas pueden trabajar en paralelo.

Replicación y tolerancia a fallos

La replicación mantiene copias de los datos en más de un lugar. La tolerancia a fallos permite continuar o recuperarse cuando una máquina o tarea falla. En nuestro proyecto local conservamos RAW y checksums; en un clúster estas responsabilidades se distribuyen y automatizan.

Batch y streaming

Nuestro script es batch: procesa archivos cerrados. Un sistema de **streaming** recibiría los eventos mientras se juega y actualizaría resultados continuamente. Herramientas como Kafka o Spark pueden intervenir en ese escenario, pero se estudiarán al profundizar en los módulos.

> DuckDB y pandas permiten aprender el proceso en un solo ordenador. Spark, Kafka, Hadoop o servicios cloud resuelven problemas semejantes cuando aumentan el volumen, la velocidad o la necesidad de disponibilidad.

Para qué nos ha servido este proceso

Hemos pasado de preparar una partida cada vez a procesar todas las partidas del lote con una sola ejecución. Ahora tenemos sus eventos reunidos en un archivo consultable, sin perder a qué partida pertenece cada uno; sabemos qué archivos se aceptaron y cuánto trabajo se realizó. Esto deja preparados los datos para futuras consultas por partida o comparaciones entre sesiones. Todavía no hemos hecho esas comparaciones ni regenerado los CSV especializados: hemos automatizado la preparación del lote. El procesamiento sigue siendo local y mantiene las tablas en memoria.

En el siguiente tema utilizaremos `all_events.parquet` para construir un dashboard: seleccionaremos una partida para ver sus cuatro paneles y calcularemos indicadores por run_id para comparar las sesiones del lote sin mezclar sus eventos.